Expected and actual declarations

Edit page 01 April 2026

Expected and actual declarations allow you to access platform-specific APIs from Kotlin Multiplatform modules. You can provide platform-agnostic APIs in the common code.

> This article describes the language mechanism of expected and actual declarations. For general recommendations on different ways to use platform-specific APIs, see Use platform-specific APIs.

Rules for expected and actual declarations

To define expected and actual declarations, follow these rules:

1. In the common source set, declare a standard Kotlin construct. This can be a function, property, class, interface, enumeration, or annotation.
2. Mark this construct with the `expect` keyword. This is your **expected declaration**. These declarations can be used in the common code, but shouldn't include any implementation. Instead, the platform-specific code provides this implementation.
3. In each platform-specific source set, declare the same construct in the same package and mark it with the `actual` keyword. This is your **actual declaration**, which typically contains an implementation using platform-specific libraries.

During compilation for a specific target, the compiler tries to match each **actual** declaration it finds with the corresponding **expected** declaration in the common code. The compiler ensures that:

- Every expected declaration in the common source set has a matching actual declaration in every platform-specific source set.
- Expected declarations don't contain any implementation.
- Every actual declaration shares the same package as the corresponding expected declaration, such as `org.mygroup.myapp.MyType`.

While generating the resulting code for different platforms, the Kotlin compiler merges the expected and actual declarations that correspond to each other. It generates one declaration with its actual implementation for each platform. Every use of the expected declaration in the common code calls the correct actual declaration in the resulting platform code.

You can declare actual declarations when you use intermediate source sets shared between different target platforms. Consider, for example, `iosMain` as an intermediate source set shared between the `iosX64Main`, `iosArm64Main`, and `iosSimulatorArm64Main` platform source sets. Only `iosMain` typically contains the actual declarations and not the platform source sets. The Kotlin compiler will then use these actual declarations to produce the resulting code for the corresponding platforms.

The IDE assists with common issues, including:

- Missing declarations
- Expected declarations that contain implementations
- Mismatched declaration signatures
- Declarations in different packages

You can also use the IDE to navigate from expected to actual declarations. Select the gutter icon to view actual declarations or use shortcuts ↗.

Different approaches for using expected and actual declarations

Let's explore the different options of using the expect/actual mechanism to solve the problem of accessing platform APIs while still providing a way to work with them in the common code.

Consider a Kotlin Multiplatform project where you need to implement the `Identity` type, which should contain the user's login name and the current process ID. The project has the `commonMain`, `jvmMain`, and `nativeMain` source sets to make the application work on the JVM and in native environments like iOS.

Expected and actual functions

You can define an `Identity` type and a factory function `buildIdentity()`, which is declared in the common source set and implemented differently in platform source sets:

1. In `commonMain`, declare a simple type and expect a factory function:

```
package identity

class Identity(val userName: String, val processID: Long)

expect fun buildIdentity(): Identity
```

2. In the `jvmMain` source set, implement a solution using standard Java libraries:

```
package identity

import java.lang.System
import java.lang.ProcessHandle

actual fun buildIdentity() = Identity(
    System.getProperty("user.name") ?: "None",
    ProcessHandle.current().pid()
)
```

3. In the `nativeMain` source set, implement a solution with POSIX ↗ using native dependencies:

```
package identity

import kotlinx.cinterop.toKString
import platform.posix.getlogin
import platform.posix.getpid

actual fun buildIdentity() = Identity(
    getlogin()?.toKString() ?: "None",
    getpid().toLong()
)
```

Here, platform functions return platform-specific `Identity` instances.

> Starting with Kotlin 1.9.0, using `getlogin()` and `getpid()` functions requires the `@OptIn` annotation.

Interfaces with expected and actual functions

If the factory function becomes too large, consider using a common `Identity` interface and implementing it differently on different platforms.

A `buildIdentity()` factory function should return `Identity`, but this time, it's an object implementing the common interface:

1. In `commonMain`, define the `Identity` interface and the `buildIdentity()` factory function:

```
// In the commonMain source set:
expect fun buildIdentity(): Identity

interface Identity {
    val userName: String
    val processID: Long
}
```

2. Create platform-specific implementations of the interface without additional use of expected and actual declarations:

```
// In the jvmMain source set:
actual fun buildIdentity(): Identity = JVMIdentity()

class JVMIdentity(
    override val userName: String = System.getProperty("user.name") ?: "
    override val processID: Long = ProcessHandle.current().pid()
) : Identity
```

```
// In the nativeMain source set:
actual fun buildIdentity(): Identity = NativeIdentity()

class NativeIdentity(
    override val userName: String = getlogin()?.toKString() ?: "None",
    override val processID: Long = getpid().toLong()
) : Identity
```

These platform functions return platform-specific `Identity` instances, which are implemented as `JVMIdentity` and `NativeIdentity` platform types.

Expected and actual properties

You can modify the previous example and expect a `val` property to store an `Identity`.

Mark this property as `expect val` and then actualize it in the platform source sets:

```
//In commonMain source set:
expect val identity: Identity

interface Identity {
    val userName: String
    val processID: Long
}
```

```
//In jvmMain source set:
actual val identity: Identity = JVMIdentity()

class JVMIdentity(
    override val userName: String = System.getProperty("user.name") ?: "no
```

```
    override val processID: Long = ProcessHandle.current().pid()
) : Identity
```

```
//In nativeMain source set:
actual val identity: Identity = NativeIdentity()

class NativeIdentity(
    override val userName: String = getlogin()?.toKString() ?: "None",
    override val processID: Long = getpid().toLong()
) : Identity
```

Expected and actual objects

When `IdentityBuilder` is expected to be a singleton on each platform, you can define it as an expected object and let the platforms actualize it:

```
// In the commonMain source set:
expect object IdentityBuilder {
    fun build(): Identity
}

class Identity(
    val userName: String,
    val processID: Long
)
```

```
// In the jvmMain source set:
actual object IdentityBuilder {
    actual fun build() = Identity(
        System.getProperty("user.name") ?: "none",
        ProcessHandle.current().pid()
    )
}
```

```
// In the nativeMain source set:
actual object IdentityBuilder {
    actual fun build() = Identity(
        getlogin()?.toKString() ?: "None",
        getpid().toLong()
    )
}
```

Recommendations on dependency injection

To create a loosely coupled architecture, many Kotlin projects adopt the dependency injection (DI) framework. The DI framework allows injecting dependencies into components based on the current environment.

For example, you might inject different dependencies in testing and in production or when deploying to the cloud compared to hosting locally. As long as a dependency is expressed through an interface, any number of different implementations can be injected, either at compile time or at runtime.

The same principle applies when the dependencies are platform-specific. In the common code, a component can express its dependencies using regular Kotlin interfaces ↗. The DI framework can then be configured to inject a platform-specific implementation, for example, from the JVM or an iOS module.

This means that expected and actual declarations are only needed in the configuration of the DI framework. See Use platform-specific APIs for examples.

With this approach, you can adopt Kotlin Multiplatform simply by using interfaces and factory functions. If you already use the DI framework to manage dependencies in your project, we recommend using the same approach for managing platform dependencies.

Expected and actual classes

> Expected and actual classes are in Beta. They are almost stable, but migration steps may be required in the future. We'll do our best to minimize any further changes for you to make.

You can use expected and actual classes to implement the same solution:

```
// In the commonMain source set:
expect class Identity() {
    val userName: String
    val processID: Int
}
```

```
// In the jvmMain source set:
actual class Identity {
    actual val userName: String = System.getProperty("user.name") ?: "None
    actual val processID: Long = ProcessHandle.current().pid()
}
```

```
// In the nativeMain source set:
actual class Identity {
    actual val userName: String = getlogin()?.toKString() ?: "None"
    actual val processID: Long = getpid().toLong()
}
```

You might have already seen this approach in demonstration materials. However, using classes in simple cases where interfaces would be sufficient is **not recommended**.

With interfaces, you don't limit your design to one implementation per target platform. Also, it's much easier to substitute a fake implementation in tests or provide multiple implementations on a single platform.

As a general rule, rely on standard language constructs wherever possible instead of using expected and actual declarations.

If you do decide to use expected and actual classes, the Kotlin compiler will warn you about the Beta status of the feature. To suppress this warning, add the following compiler option to your Gradle build file:

```
kotlin {
    compilerOptions {
        // Common compiler options applied to all Kotlin source sets
        freeCompilerArgs.add("-Xexpect-actual-classes")
    }
}
```

Inheritance from platform classes

There are special cases when using the `expect` keyword with classes may be the best approach. Let's say that the `Identity` type already exists on the JVM:

```
open class Identity {
    val login: String = System.getProperty("user.name") ?: "none"
    val pid: Long = ProcessHandle.current().pid()
}
```

To fit it in the existing codebase and frameworks, your implementation of the `Identity` type can inherit from this type and reuse its functionality:

1. To solve this problem, declare a class in `commonMain` using the `expect` keyword:

   ```
   expect class CommonIdentity() {
       val userName: String
       val processID: Long
   }
   ```

2. In `nativeMain`, provide an actual declaration that implements the functionality:

   ```
   actual class CommonIdentity {
       actual val userName = getlogin()?.toKString() ?: "None"
       actual val processID = getpid().toLong()
   }
   ```

3. In `jvmMain`, provide an actual declaration that inherits from the platform-specific base class:

   ```
   actual class CommonIdentity : Identity() {
       actual val userName = login
   ```

```
    actual val processID = pid
}
```

Here, the `CommonIdentity` type is compatible with your own design while taking advantage of the existing type on the JVM.

Application in frameworks

As a framework author, you can also find expected and actual declarations useful for your framework.

If the example above is part of a framework, the user has to derive a type from `CommonIdentity` to provide a display name.

In this case, the expected declaration is abstract and declares an abstract method:

```
// In commonMain of the framework codebase:
expect abstract class CommonIdentity() {
    val userName: String
    val processID: Long
    abstract val displayName: String
}
```

Similarly, actual implementations are abstract and declare the `displayName` method:

```
// In nativeMain of the framework codebase:
actual abstract class CommonIdentity {
    actual val userName = getlogin()?.toKString() ?: "None"
    actual val processID = getpid().toLong()
    actual abstract val displayName: String
}
```

```
// In jvmMain of the framework codebase:
actual abstract class CommonIdentity : Identity() {
    actual val userName = login
    actual val processID = pid
    actual abstract val displayName: String
}
```

The framework users need to write common code that inherits from the expected declaration and implement the missing method themselves:

```
// In commonMain of the users' codebase:
class MyCommonIdentity : CommonIdentity() {
    override val displayName = "Admin"
}
```

Advanced use cases

There are a number of special cases regarding expected and actual declarations.

Using type aliases to satisfy actual declarations

The implementation of an actual declaration does not have to be written from scratch. It can be an existing type, such as a class provided by a third-party library.

You can use this type as long as it meets all the requirements associated with the expected declaration. For example, consider these two expected declarations:

```
expect enum class Month {
    JANUARY, FEBRUARY, MARCH, APRIL, MAY, JUNE, JULY,
    AUGUST, SEPTEMBER, OCTOBER, NOVEMBER, DECEMBER
}

expect class MyDate {
    fun getYear(): Int
    fun getMonth(): Month
    fun getDayOfMonth(): Int
}
```

Within a JVM module, the `java.time.Month` enum can be used to implement the first expected declaration and the `java.time.LocalDate` class to implement the second. However, there's no way to add the `actual` keyword directly to these types.

Instead, you can use type aliases ↗ to connect the expected declarations and the platform-specific types:

```
actual typealias Month = java.time.Month
actual typealias MyDate = java.time.LocalDate
```

In this case, define the `typealias` declaration in the same package as the expected declaration and create the referred class elsewhere.

> Since the `LocalDate` type uses the `Month` enum, you need to declare both of them as expected classes in the common code.

Expanded visibility in actual declarations

You can make actual implementations more visible than the corresponding expected declaration. This is useful if you don't want to expose your API as public for common clients.

Currently, the Kotlin compiler issues an error in the case of visibility changes. You can suppress this error by applying `@Suppress("ACTUAL_WITHOUT_EXPECT")` to the actual type

alias declaration. Starting with Kotlin 2.0, this limitation will not apply.

For example, if you declare the following expected declaration in the common source set:

```
internal expect class Messenger {
    fun sendMessage(message: String)
}
```

You can use the following actual implementation in a platform-specific source set as well:

```
@Suppress("ACTUAL_WITHOUT_EXPECT")
public actual typealias Messenger = MyMessenger
```

Here, an internal expected class has an actual implementation with an existing public `MyMessenger` using type aliases.

Additional enumeration entries on actualization

When an enumeration is declared with `expect` in the common source set, each platform module should have a corresponding `actual` declaration. These declarations must contain the same enum constants, but they can also have additional constants.

This is useful when you actualize an expected enum with an existing platform enum. For example, consider the following enumeration in the common source set:

```
// In the commonMain source set:
expect enum class Department { IT, HR, Sales }
```

When you provide an actual declaration for `Department` in platform source sets, you can add extra constants:

```
// In the jvmMain source set:
actual enum class Department { IT, HR, Sales, Legal }
```

```
// In the nativeMain source set:
actual enum class Department { IT, HR, Sales, Marketing }
```

However, in this case, these extra constants in the platform source sets won't match with those in the common code. Therefore, the compiler requires you to handle all additional cases.

The function that implements the `when` construction on `Department` requires an `else` clause:

```
// An else clause is required:
fun matchOnDepartment(dept: Department) {
    when (dept) {
        Department.IT -> println("The IT Department")
        Department.HR -> println("The HR Department")
        Department.Sales -> println("The Sales Department")
        else -> println("Some other department")
    }
}
```

Expected annotation classes

Expected and actual declarations can be used with annotations. For example, you can declare an `@XmlSerializable` annotation, which must have a corresponding actual declaration in each platform source set:

```
@Target(AnnotationTarget.CLASS)
@Retention(AnnotationRetention.RUNTIME)
expect annotation class XmlSerializable()

@XmlSerializable
class Person(val name: String, val age: Int)
```

It might be helpful to reuse existing types on a particular platform. For example, on the JVM, you can define your annotation using the existing type from the JAXB specification ↗:

```
import javax.xml.bind.annotation.XmlRootElement

actual typealias XmlSerializable = XmlRootElement
```

There is an additional consideration when using `expect` with annotation classes. Annotations are used to attach metadata to code and do not appear as types in signatures. It's not essential for an expected annotation to have an actual class on a platform where it's never required.

You only need to provide an `actual` declaration on platforms where the annotation is used. This behavior isn't enabled by default, and it requires the type to be marked with `OptionalExpectation` ↗.

Take the `@XmlSerializable` annotation declared above and add `OptionalExpectation`:

```
@OptIn(ExperimentalMultiplatform::class)
@Target(AnnotationTarget.CLASS)
@Retention(AnnotationRetention.RUNTIME)
@OptionalExpectation
expect annotation class XmlSerializable()
```

If an actual declaration is missing on a platform where it's not required, the compiler won't generate an error.

What's next?

For general recommendations on different ways to use platform-specific APIs, see Use platform-specific APIs.